



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS
ICS2121 - MÉTODOS DE OPTIMIZACIÓN - 2025-2

Profesor: JORGE VERA

Ayudantes: - CHRISTOPH BÜRGER (CHRISTOPH.BUERGER@UC.CL)
- FRANCISCO TAMARÍN (FTAMARIN@UC.CL)
- ALBERTO URETA (ALBERTO.URETA@UC.CL)

Taller 2

Método del Gradiente y Newton

14 de agosto de 2025

Resumen

■ Gradiente

El Gradiente de una función $f : \mathbb{R}^n \rightarrow \mathbb{R}$ se define como sigue:

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f(x)}{\partial x_1} \\ \frac{\partial f(x)}{\partial x_2} \\ \vdots \\ \frac{\partial f(x)}{\partial x_n} \end{pmatrix}$$

■ Matriz Hessiana

Sea una función $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Se define la matriz hessiana como sigue:

$$\nabla^2 f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

- **Dirección de descenso:** d es dirección de descenso si cumple que $d^T \nabla f(x) < 0$

■ **Método de Gradiente (Convergencia lineal):**

1. Sea $x^0 \in \mathbb{R}^n, k = 0, \epsilon > 0$.
2. Sea $d^k = -\nabla f(x^k)$
3. Si $\|\nabla f(x^k)\| \leq \epsilon$, entonces STOP.
4. Sea λ^k solución de $\min_{\lambda \geq 0} f(x^k + \lambda d^k)$
5. Sea $x^{k+1} = x^k + \lambda^k d^k, k \leftarrow k + 1$, go to 2.

■ **Método de Newton (Convergencia cuadrática):**

1. Sea $x^0 \in \mathbb{R}^n, k = 0, \epsilon > 0$.
2. Sea $d^k = -[\nabla^2 f(x^k)]^{-1} \nabla f(x^k)$
3. Si $\|\nabla f(x^k)\| \leq \epsilon$, entonces STOP.
4. Sea λ^k solución de $\min_{\lambda \geq 0} f(x^k + \lambda d^k)$
5. Sea $x^{k+1} = x^k + \lambda^k d^k, k \leftarrow k + 1$, go to 2.

Ejercicio 1

Se está realizando una investigación sobre los factores que influyen en qué personas de una cierta población adquieran una enfermedad. Lo que se busca es relacionar la propensión al contagio con algunos factores como peso, edad, presión arterial y otros que podrían ser explicativos. Para esto, se dispone de datos de m personas y se ha definido un indicador y donde $y_i = 1$ si la persona contrajo la enfermedad e $y_i = 0$ en caso contrario. Además, se dispone de mediciones de n variables (edad, peso, etc.) para esas m personas, donde u_{ij} es el valor de la variable j para la persona i . Se tiene que $m > n$. Además, en el grupo bajo estudio hay q personas contagiadas y $m - q$ no contagiadas y los datos están ordenados de modo que los contagiados corresponden a las personas $i = 1, \dots, q$ y los no contagiados corresponden a las personas $i = q + 1, \dots, m$. Para este tipo de estudios se puede usar un “modelo logístico”, en el cual se modela la probabilidad de adquirir la enfermedad, p , de la siguiente forma:

$$p = \frac{e^{\alpha^T u}}{1 + e^{\alpha^T u}}$$

donde $\alpha \in \mathbb{R}^n$ corresponde a los parámetros del modelo (a veces hay, además, un término constante pero acá lo estamos obviando para simplificar). Entonces, para la persona i , se estima una probabilidad dada por

$$p_i = \frac{e^{\alpha^T u^i}}{1 + e^{\alpha^T u^i}}, \quad i = 1, \dots, m$$

donde $u^i \in \mathbb{R}^n$ corresponde al vector de las observaciones para la persona i . Es decir, las componentes de u^i son $u_{i1}, u_{i2}, \dots, u_{in}$.

Este problema se aborda buscando los estimadores de “máxima verosimilitud” del modelo, es decir, los valores de los parámetros α que maximizan la probabilidad conjunta de contagio y no contagio en el grupo observado, que es:

$$\prod_{i=1}^q p_i \prod_{i=q+1}^m (1 - p_i)$$

Como esta expresión es compleja, lo que se hace, en forma equivalente, es maximizar la log-verosimilitud (con logaritmo natural) dada por:

$$l(\alpha) = \sum_{i=1}^q \alpha^T u^i - \sum_{i=1}^m \log(1 + e^{\alpha^T u^i}) = \sum_{i=1}^q \sum_{j=1}^n \alpha_j u_{ij} - \sum_{i=1}^m \log(1 + e^{\sum_{j=1}^n \alpha_j u_{ij}})$$

(la última fórmula está bien, la primera suma sí es desde 1 a q .) Esta es la función de n variables con la que trabajamos y lo que queremos hacer es resolver el siguiente problema de optimización:

$$\max_{\alpha \in \mathbb{R}^n} l(\alpha)$$

(a) Determine $\nabla l(\alpha)$

Solución: Las primeras derivadas parciales, para cada j son:

$$\frac{\partial l(\alpha)}{\partial \alpha_j} = \sum_{i=1}^q u_{ij} - \sum_{i=1}^m \frac{u_{ij} e^{\sum_{j=1}^n \alpha_j u_{ij}}}{1 + e^{\sum_{j=1}^n \alpha_j u_{ij}}} = \sum_{i=1}^q u_{ij} - \sum_{i=1}^m \frac{u_{ij} e^{\alpha^T u^i}}{1 + e^{\alpha^T u^i}} \quad j = 1, \dots, n$$

Y con eso se determina el Gradiente de la función $l(\alpha)$.

(b) Determine $\nabla^2 l(\alpha)$

Solución: Las segundas derivadas son:

$$\frac{\partial^2 l(\alpha)}{\partial \alpha_j \partial \alpha_k} = - \sum_{i=1}^m \frac{u_{ij} u_{ik} e^{\alpha^T u^i} (1 + e^{\alpha^T u^i}) - u_{ij} u_{ik} e^{\alpha^T u^i} e^{\alpha^T u^i}}{(1 + e^{\alpha^T u^i})^2} = - \sum_{i=1}^m \frac{u_{ij} u_{ik} e^{\alpha^T u^i}}{(1 + e^{\alpha^T u^i})^2} \quad j, k = 1, \dots, n$$

Y con eso se puede armar el Hessiano de $l(\alpha)$.

(c) Supongamos que queremos llegar a una solución con un error $\epsilon > 0$. Estime el esfuerzo computacional total (en flops) para llegar a esa meta. (Aquí puede asumir conocidas las constantes “escondidas” en algunas expresiones, denótelas como algún valor constante). Explique con claridad cualquier supuesto adicional o estimación que haga.

Solución: Notamos que tanto para el Gradiente como para el hessiano debemos calcular $e^{\alpha^T u^i}$ para cada $i = 1, \dots, m$ en las sumatorias, como calcular $e^{\alpha^T u^i}$ toma $\mathcal{O}(n)$ flops para cada i , todos estos valores los podemos precomputar al inicio de cada iteración en $\mathcal{O}(nm)$ flops.

Luego para el Gradiente tenemos que $\sum_{i=1}^q u_{ij}$ es un valor fijo que lo podemos precomputar de manera inicial y como $e^{\alpha^T u^i}$ están precomputados de manera inicial para cada iteración $\sum_{i=1}^q u_{ij} - \sum_{i=1}^m \frac{u_{ij} e^{\alpha^T u^i}}{1 + e^{\alpha^T u^i}}$ se puede calcular de manera eficiente en $\mathcal{O}(m)$ flops, para cada componente j del Gradiente, por lo que calcular el Gradiente de manera eficiente se puede realizar en $\mathcal{O}(nm)$ flops. Calcular el Gradiente es lo más costoso del método del Gradiente, por lo que podemos asumir un costo operacional por iteración de $\mathcal{O}(nm)$ flops para el método del Gradiente ya que la etapa de linersearch la podemos asumir que es equivalente a calcular un número constante de veces el Gradiente, por lo que el costo por iteración sigue siendo equivalente a $\mathcal{O}(nm)$.

Para Newton, utilizando que $e^{\alpha^T u^i}$ están precalculados (en el Gradiente) de manera inicial para cada iteración para cada i , podemos aprovechar esto para calcular cada componente del hessiano en $\mathcal{O}(m)$ flops. Luego como el Hessiano es una matriz de $n \times n$, entonces computar el hessiano toma $\mathcal{O}(n^2 m)$ flops y luego invertir el hessiano toma

$\mathcal{O}(n^3)$, por lo que por iteración el método de Newton toma $\mathcal{O}(n^3 + n^2m)$. (Notamos que lineasearch es menos costoso que obtener e invertir el Hessiano, así que no le sumamos costo por iteración).

Por último nos debemos fijar en la convergencia de los métodos, donde el método del Gradiente tiene una convergencia de $\mathcal{O}(\log(\frac{1}{\epsilon}))$ iteraciones para llegar a una solución con error $\epsilon > 0$. El método de Newton requiere de $\mathcal{O}(\log \log(\frac{1}{\epsilon}))$ iteraciones.

Por lo que el esfuerzo computacional total para los dos métodos es:

Gradiente: $\mathcal{O}(nm \log(\frac{1}{\epsilon}))$

Newton: $\mathcal{O}((n^3 + n^2m) \log \log(\frac{1}{\epsilon}))$

- (d) Considere que un amigo que ya dio el ramo le dice que siempre es conveniente usar el método de Newton para encontrar una solución ϵ -óptima. Argumente si lo mencionado por su amigo es correcto.

Solución: La afirmación del amigo no es correcta.

Si bien el método de Newton puede alcanzar convergencia cuadrática cuando se cumplen ciertas condiciones, no siempre es la mejor opción para encontrar una solución ϵ -óptima.

Esto se debe principalmente a el costo computacional por iteración: Newton requiere calcular e invertir el Hessiano en cada paso, lo que cuesta $\mathcal{O}(n^3 + n^2m)$, mientras que el método del Gradiente es $\mathcal{O}(nm)$. Esto hace que Newton sea poco atractivo si n o m es muy grande y no se compensa la velocidad del algoritmo.

- (e) Suponga que le dicen que $n \gg m$, encuentre a partir de cual n es más conveniente usar Gradiente sobre Newton para la obtención de una solución ϵ -óptima (en términos de tiempo total de cómputo). Para esto puede asumir que $\mathcal{O}(X) = X$, es decir, todas las constantes ocultas son 1.

Solución: Recordemos de la pregunta c) que el esfuerzo computacional total para los dos métodos es:

Gradiente: $\mathcal{O}(nm \log(\frac{1}{\epsilon}))$

Newton: $\mathcal{O}((n^3 + n^2m) \log \log(\frac{1}{\epsilon}))$

Como a nosotros nos piden encontrar a partir de cual n es conveniente usar Gradiente sobre Newton, busquemos, usando el supuesto que $\mathcal{O}(X) = X$:

$$(nm \log(\frac{1}{\epsilon})) < ((n^3 + n^2m) \log \log(\frac{1}{\epsilon}))$$

$$(m \log(\frac{1}{\epsilon})) < ((n^2 + nm) \log \log(\frac{1}{\epsilon}))$$

Pero como $n \gg m$, las expresiones se pueden simplificar a:

$$(\log(\frac{1}{\epsilon})) < ((n^2) \log \log(\frac{1}{\epsilon})) \implies n > (\frac{\log \frac{1}{\epsilon}}{\log \log \frac{1}{\epsilon}})^{1/2}$$

Ejercicio 2

Considere la siguiente función de máxima verosimilitud para un problema de estimación estadística:

$$g(v_1, \dots, v_k) = \sum_{i=1}^m \left(y_i \log \left(\sum_{j=1}^k v_j u_j^i \right) - \sum_{j=1}^k v_j u_j^i \right)$$

donde los y_i son datos diarios de accidentes de tránsito y los u_j^i son valores de varios factores explicativos que se postulan. El problema consiste en determinar v de modo que se maximice $g(v)$.

El Gradiente y el Hessiano de esa función en un punto arbitrario x se puede calcular sin mucha dificultad. Las fórmulas son las siguientes:

$$\nabla g(v) = \sum_{i=1}^m \left(y_i \frac{u^i}{v^T u^i} - u^i \right), \quad \nabla^2 g(v) = \sum_{i=1}^m \left(-y_i \frac{u^i (u^i)^T}{(v^T u^i)^2} \right)$$

Donde llamamos U a la matriz de los datos de los factores explicativos, siendo u^i el vector de la fila i de la matriz de datos de los factores explicativos.

- (a) Determine el orden de número de operaciones aritméticas (flops) que se tienen que realizar para evaluar el Gradiente, en función del número de factores explicativos k , y del número de observaciones m .

Solución: Para calcular la complejidad del Gradiente se pueden seguir los siguientes pasos:

1. $p = v^T u^i, \mathcal{O}(k)$
2. $q = \frac{u^i}{p}, \mathcal{O}(k)$
3. $r = y_i \cdot q, \mathcal{O}(k)$
4. $s^i = r - u^i, \mathcal{O}(k)$
5. $t = \sum_{i=1}^m s, \mathcal{O}(mk)$

Por lo tanto, la complejidad de calcular el Gradiente es $\mathcal{O}(mk)$.

- (b) Sobre la base de lo anterior, determine el orden del costo por iteración para el método del Gradiente.

Solución: La complejidad del método del Gradiente vendrá dada por calcular la dirección de descenso, es decir, $-\nabla g(v)$.

-
- (c) Ahora determine el orden de número de operaciones aritméticas (flops) que se tienen que realizar para evaluar el hessiano, en función del número de factores explicativos k y del número de observaciones m .

Solución: Para calcular la complejidad del hessiano se pueden seguir los siguientes pasos:

1. $p = v^T u^i, \mathcal{O}(k)$
2. $b = p^2, \mathcal{O}(1)$
3. $q = u^i (u^i)^T, \mathcal{O}(k^2)$
4. $r = \frac{q}{b}, \mathcal{O}(k^2)$
5. $s^i = -y_i \cdot r, \mathcal{O}(k^2)$
6. $t = \sum_{i=1}^m s, \mathcal{O}(mk^2)$

Por lo tanto, la complejidad de calcular el hessiano es $\mathcal{O}(mk^2)$.

- (d) Sobre la base de lo anterior, determine el orden del costo por iteración para el método de Newton.

Solución: La complejidad del método de Newton vendrá dada por calcular el hessiano ($\mathcal{O}(mk^2)$) e invertirlo ($\mathcal{O}(k^3)$). Con lo que la complejidad del método queda $\mathcal{O}(k^3 + mk^2)$.

- (e) Determine, basado en sus respuestas anteriores, un criterio que permita decir a partir de qué cantidad de factores (k) o de observaciones (m) conviene más usar el Método del Gradiente que el Método de Newton para llegar en menos tiempo a una solución ϵ -aproximada. Para hacer su estimación asuma, en forma simplificada, que las constantes “ocultas” en las expresiones $\mathcal{O}(\cdot)$ son todas iguales a 1. Justifique con precisión todo supuesto adicional que haga para su análisis (Nota: en su análisis tendrá que ver si sus criterios se aplican sólo a k o sólo a m o a ambos).

Solución: Para llegar a un error ϵ se necesita,

En el método del Gradiente: $K_1 \log \left(\frac{1}{\epsilon} \right)$ iteraciones.

En el método de Newton: $K_2 \log \log \left(\frac{1}{\epsilon} \right)$ iteraciones.

Luego, sea T_1 y T_2 el tiempo total por el número de iteraciones para el método del

Gradiente y el método de Newton,

$$T_1 = C_1(mk)K_1 \log \left(\frac{1}{\epsilon} \right) \quad (1)$$

$$T_2 = C_2(k^3 + mk^2)K_2 \log \log \left(\frac{1}{\epsilon} \right) \quad (2)$$

Del enunciado sabemos que podemos asumir que todas las constantes “ocultas” en las expresiones $\mathcal{O}(\cdot)$ son iguales a 1, es decir, $K_1 = K_2 = C_1 = C_2 = 1$. Para que convenga usar el método del Gradiente al método de Newton se debe cumplir,

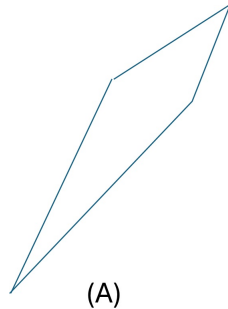
$$T_1 \leq T_2 \quad (3)$$

$$(mk) \log \left(\frac{1}{\epsilon} \right) \leq (k^3 + mk^2) \log \log \left(\frac{1}{\epsilon} \right) \quad (4)$$

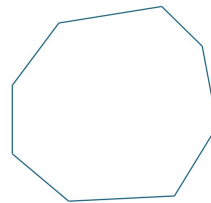
$$\frac{mk}{k^3 + mk^2} \leq \frac{\log \log \left(\frac{1}{\epsilon} \right)}{\log \left(\frac{1}{\epsilon} \right)} \quad (5)$$

Cuando se cumple este criterio se preferirá usar el método del Gradiente al método de Newton. En el caso contrario se optará por usar el método de Newton. Se puede notar que el criterio depende tanto de m como de k . En el análisis del criterio se pueden mencionar más cosas, pero se espera que cómo mínimo, se mencione que ambos factores influyen.

- (f) **(Propuesto)** Ahora queremos usar el Método del Gradiente para abordar el problema. Las siguientes figuras muestran (en dos dimensiones) dos posibles poliedros en los que se querría resolver el problema: el poliedro (A) y el poliedro (B).



(A)

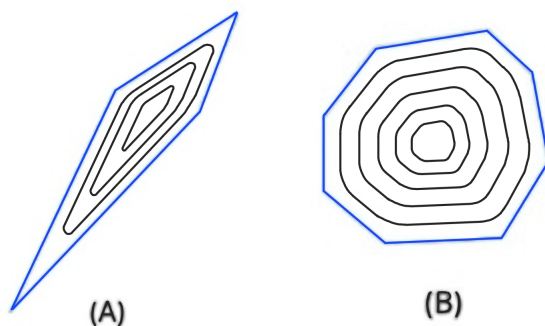


(B)

¿En cuál de los dos casos espera que el método del Gradiente converja en menos iteraciones? (suponiendo que parten de un punto más o menos alejado del centro). Justifique con precisión su respuesta sobre la base de los contenidos del curso.

Solución: Se espera una convergencia más rápida para el poliedro B . Para justificar esto, es necesario notar que, en este caso, como la función objetivo está determinada

por el mismo dominio, y lo que busca es alejarnos de sus caras, luego las curvas de nivel de la función f tienen una forma muy parecida al poliedro:



Recordamos que la convergencia del método del Gradiente depende del grado de excentricidad de las curvas de nivel. Por lo tanto, como B presenta menor excentricidad, es esperable una convergencia más rápida.

Nota: La imagen es solo una referencia y no representa en exactitud las curvas de nivel de la función.

Ejercicios Propuestos

Ejercicio 1

Vamos a considerar el siguiente problema, que es relevante en diversas aplicaciones en geometría computacional. Tenemos un poliedro definido por un conjunto de desigualdades lineales, de la forma

$$P = \{x \in \mathbb{R}^n : a_i^T x \leq b_i, i = 1, \dots, m\}$$

donde $a_i \in \mathbb{R}^n$, $b_i \in \mathbb{R}$, para $i = 1, \dots, m$, $m > n$. Vamos a asumir, también, que P tiene interior no vacío, es decir, existe un punto $x^0 \in \mathbb{R}^n$ tal que $a_i^T x^0 < b_i, i = 1, \dots, m$. Interesa encontrar un punto en el interior de este conjunto, que esté separado de las hiperplanos definidos por las restricciones. Para esto, se plantea la siguiente función, definida en el interior del conjunto P :

$$f(x) = \sum_{i=1}^m \frac{1}{b_i - a_i^T x}$$

Note que cuando x se acerca a alguno de los hiperplanos de la frontera de P , los que están definidos por $a_i^T x = b_i$, entonces alguno de los términos $b_i - a_i^T x$ se hace chico, con lo cual su recíproco se hace grande y el valor de $f(x)$ se hace grande. Por lo tanto, cuando x se acerca a la frontera de P desde adentro, f aumenta de valor y se va a ∞ a medida que x se acerca más y más al borde, y no está definida en la frontera. Se propone, entonces, resolver el siguiente problema de optimización para encontrar el punto “más al centro” de P : $\min f(x)$. Si bien es verdad que el problema debería restringirse al interior de P , no es realmente necesario ya que, en la medida que los algoritmos que conocemos partan desde un punto en el interior (por ejemplo, x^0), los puntos generados por las iteraciones deberían permanecer dentro de P ya que la función objetivo crece al acercarse a la frontera. Le comentamos también que la función f es convexa, si se considera solamente dentro del interior del conjunto P .

- (a) Desarrolle fórmulas explícitas para el Gradiente y el Hessiano de la función f .

Solución: Tenemos las fórmulas para una componente genérica del Gradiente y hessiano, respectivamente,

$$[\nabla f(x)]_j = \sum_{i=1}^m \frac{a_{ij}}{(b_i - a_i^T x)^2}$$

$$[\nabla^2 f(x)]_{jk} = \sum_{i=1}^m \frac{2a_{ij}a_{ik}}{(b_i - a_i^T x)^3}$$

donde $[\nabla f(x)]_j$ es la j -ésima coordenada del Gradiente de f , y $[\nabla^2 f(x)]_{jk}$ es la entrada de la j -ésima fila, k -ésima columna, del Hessiano de f .

- (b) Estime el esfuerzo computacional, en flops, para evaluar el Gradiente y Hessiano de la función f en algún punto de x , en función de m y n . Debe hacer las estimaciones considerando una evaluación eficiente de las fórmulas que desarrolle en la parte a).

Solución: Notamos que tanto en el Gradiente como en el Hessiano de la función, se calcula de manera repetitiva la siguiente operación:

$$b_i - a_i^T x \quad \forall i = 1, \dots, m$$

Luego, como debemos hacer una evaluación eficiente de $\nabla f(x)$ y $\nabla^2 f(x)$, debemos precomputar lo siguiente:

$$c = b - Ax$$

Notamos entonces que $c_i = b_i - a_i^T x$. Calcular el vector c toma un esfuerzo computacional de $\mathcal{O}(mn)$ por el producto entre la matriz $A \in \mathbb{R}^{m \times n}$ y el vector $x \in \mathbb{R}^n$. El resto de operaciones se desglosa a continuación:

• **Gradiente:**

$$[\nabla f(x)]_j = \sum_{i=1}^m \frac{a_{ij}}{c_i^2}$$

- 1) $\alpha_i^{(j)} := a_{ij}/c_i^2$: Toma $\mathcal{O}(1)$ operaciones.
- 2) $\sum_{i=1}^m \alpha_i^{(j)}$: Son m operaciones de costo $\mathcal{O}(1)$, luego toma $\mathcal{O}(m)$
- 3) El Gradiente tiene n coordenadas, cada una con costo $\mathcal{O}(m)$, luego el Gradiente en total toma $\mathcal{O}(mn)$ operaciones.

• **Hessiano:**

$$[\nabla^2 f(x)]_{jk} = \sum_{i=1}^m \frac{2a_{ij}a_{ik}}{c_i^3}$$

- 1) $\alpha_i^{(jk)} := 2a_{ij}a_{ik}/c_i^3$: Toma $\mathcal{O}(1)$ operaciones.
- 2) $\sum_{i=1}^m \alpha_i^{(jk)}$: son m operaciones de costo $\mathcal{O}(1)$, luego toma $\mathcal{O}(m)$ operaciones.
- 3) El Hessiano tiene n^2 entradas con costo $\mathcal{O}(m)$, luego toma $\mathcal{O}(mn^2)$ operaciones.

- (c) Supongamos que decidimos usar el Método de Newton para este problema y queremos llegar a una solución con un error $\epsilon > 0$. Estime el esfuerzo computacional total (en flops) para llegar a esa meta, en función de m y n . Asuma que la etapa de Linesearch requiere un esfuerzo fijo de $p < m$ iteraciones en las que se debe evaluar el Gradiente de f en distintos puntos. Su expresión final puede quedar en términos de m , n , p , y ϵ , además de constantes. (Aquí puede asumir conocidas las constantes “escondidas” en algunas expresiones, denótelas con un símbolo adecuado). Explique con claridad cualquier supuesto adicional o estimación que haga.

Solución: Estimemos primero cuántas iteraciones realizará el algoritmo. Sabemos que el método de Newton tiene convergencia de $\mathcal{O}(\log(\log(1/\varepsilon)))$. Luego, se realizan:

$$k = \kappa_1 \log(\log(1/\varepsilon))$$

iteraciones, con κ_1 una constante conocida. Estimemos ahora el esfuerzo computacional por iteración. En cada iteración debemos realizar los siguientes pasos:

- 1) Computar $\nabla f(x^k)$: Toma $\kappa_2 mn$ operaciones, con κ_2 una constante conocida.
- 2) Computar $\nabla^2 f(x^k)$: Toma $\kappa_3 mn^2$ operaciones, con κ_3 una constante conocida.
- 3) Invertir el Hessiano: Invertir una matriz $H \in \mathbb{R}^{n \times n}$ es un algoritmo de orden $\mathcal{O}(n^3)$. Luego, toma $\kappa_4 n^3$ operaciones, con κ_4 una constante conocida.
- 4) Computar $[\nabla^2 f(x^k)]^{-1} \cdot \nabla f(x^k)$: Esta operación matriz-vector toma $\kappa_5 n^2$ operaciones, con κ_5 una constante conocida.
- 5) Linesearch: Toma p iteraciones en la que se calcula el Gradiente, luego toma $\kappa_6 pmn$ operaciones, con κ_6 constante conocida.
- 6) Computar $\lambda \cdot d^k$: Es una multiplicación de escalar por vector. Toma $\kappa_7 n$ operaciones, con κ_7 constante conocida.

Finalmente, el método realizará una cantidad de operaciones igual a:

$$\kappa_1 \log(\log(1/\varepsilon)) \cdot ((\kappa_2 + \kappa_6 p)mn + \kappa_3 mn^2 + \kappa_4 n^3 + \kappa_5 n^2 + \kappa_7 n)$$

Ejercicio 2

Nos interesa encontrar una solución al sistema de ecuaciones $Ax = b$, donde $A \in \mathbb{R}^{m \times n}$ y $b \in \mathbb{R}^m$, con $m < n$. También asumiremos que las filas de A son linealmente independientes. Como sabemos, un sistema de este tipo tiene muchas soluciones posibles. Una forma de “regularizar” el problema, diferente a lo que vimos en clases, es pedir que la solución x que se encuentre tenga “poca variación”. Esto se usa, por ejemplo, en aplicaciones de análisis de señales. Explicamos esto a continuación:

Sea $x \in \mathbb{R}^n$. Se define la “variación” del vector $x \in \mathbb{R}^n$ como el siguiente vector $\Delta(x) \in \mathbb{R}^{n-1}$ dado por:

$$\Delta(x)_j = x_{j+1} - x_j, j = 1, \dots, n-1$$

Entonces, vamos a plantear resolver el siguiente problema de optimización:

$$\min_{x \in \mathbb{R}^n} \tau \|\Delta(x)\|_2^2 + \|Ax - b\|_2^2$$

donde τ es un parámetro que busca un balance entre las dos partes de la función objetivo.

- (a) Muestre que $\Delta(x)$ se puede escribir de la forma $\Delta(x) = Bx$ donde $B \in \mathbb{R}^{(n-1) \times n}$ y determine explícitamente esa matriz B .

Solución: Notamos que $\Delta(x)_j = -x_j + x_{j+1}$. Luego si definimos e_j como el j -ésimo vector canónico en $\mathbb{R}^n$. Por lo que:

$$\Delta(x)_j = -e_j^T x + e_{j+1}^T x = (-e_j^T + e_{j+1}^T)x$$

Luego podemos notar que:

$$B = \begin{pmatrix} -e_1^T + e_2^T \\ -e_2^T + e_3^T \\ \vdots \\ -e_{n-1}^T + e_n^T \end{pmatrix}$$

O equivalentemente

$$B = \begin{pmatrix} -1 & 1 & 0 & \dots & 0 & 0 & 0 \\ 0 & -1 & 1 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & -1 & 1 & 0 \\ 0 & 0 & 0 & \dots & 0 & -1 & 1 \end{pmatrix}$$

Donde $B \in \mathbb{R}^{(n-1) \times n}$

-
- (b) Según a), entonces, la función objetivo del problema es de la forma $f(x) = \tau\|Bx\|_2^2 + \|Ax - b\|_2^2$. También es fácil ver que esta función es convexa (es suma de términos cuadráticos convexos). Determine el Gradiente y el Hessiano de f .

Solución: Se tiene

$$\begin{aligned}
 f(x) &= \tau\|Bx\|_2^2 + \|Ax - b\|_2^2 \\
 &= \tau(Bx)^T(Bx) + (Ax - b)^T(Ax - b) \\
 &= \tau(x^T B^T Bx) + (x^T A - b^T)(Ax - b) \\
 &= \tau(x^T B^T Bx) + x^T A^T Ax - x^T A^T b - b^T Ax + b^T b
 \end{aligned}$$

Luego

$$\begin{aligned}
 \nabla f(x) &= 2\tau B^T Bx + 2A^T Ax - A^T b - A^T b \\
 &= 2\tau B^T Bx + 2A^T Ax - 2A^T b \\
 &\Rightarrow \nabla^2 f(x) = 2\tau B^T B + 2A^T A
 \end{aligned}$$

- (c) Determine explícitamente la solución óptima del problema $\min f(x)$. ¿Cuánto es el esfuerzo computacional (en “flops”) para encontrar esta solución?

Solución: Como la función es convexa basta con encontrar x^* tal que $\nabla f(x^*) = 0$

$$\begin{aligned}
 \nabla f(x^*) &= 2\tau B^T Bx^* + 2A^T Ax^* - 2A^T b = 0 \\
 &\Rightarrow \tau B^T Bx^* + A^T Ax^* = A^T b \\
 &\Rightarrow (\tau B^T B + A^T A)x^* = A^T b \\
 &\Rightarrow x^* = (\tau B^T B + A^T A)^{-1} A^T b
 \end{aligned}$$

Luego se puede realizar las operaciones en el siguiente orden:

- a) $\tau B^T B$ $\mathcal{O}(n^3)$
- b) $A^T A$ $\mathcal{O}(n^2 m)$
- c) $(\tau B^T B + A^T A)^{-1}$ $\mathcal{O}(n^3)$
- d) $A^T b$ $\mathcal{O}(nm)$
- e) $(\tau B^T B + A^T A)^{-1} A^T b$ $\mathcal{O}(n^2)$

Como se cumple que $m < n$ la operación de mayor esfuerzo es invertir la matriz en (3.). Por lo que el esfuerzo computacional para encontrar la solución es $\mathcal{O}(n^3)$

-
- (d) En forma alternativa a hacer lo de (c), se propone usar el Método del Gradiente para abordar el problema. Si el algoritmo se implementa de forma eficiente, ¿Cuál es el esfuerzo computacional (en flops) de una iteración del Método del Gradiente? (Piense bien esto, fíjese en cosas que se deben hacer en cada iteración, que es lo que estamos preguntando, frente a otras que pueden hacerse una sola vez al comienzo y que no las incluimos en el costo por iteración).

Solución: Por lo visto en clase el método del Gradiente tiene un costo operacional por iteración de $\mathcal{O}(n)$, sin embargo esto no considera el costo de obtener el vector Gradiente o el costo por resolver Linesearch. Por ahora omitiremos el costo de resolver Linesearch y observemos el costo de obtener el Gradiente en cada iteración:

$$\nabla f(x^k) = 2\tau B^T B x^k + 2A^T A x^k - 2A^T b$$

Notamos que podemos precomputar y guardar las matrices $2\tau B^T B$ y $2A^T A$ y el vector resultante de $2A^T b$, por lo que en cada iteración simplemente resolvemos la multiplicaciones de matrices con vectores

- a) $(2A^T A)x^k \mathcal{O}(n^2)$
- b) $(2\tau B^T B)x^k \mathcal{O}(n^2)$
- c) $(2\tau B^T B)x^k + (2A^T A)x^k - 2A^T b \mathcal{O}(n)$

Por lo que el esfuerzo computacional por iteración si se evitan las multiplicaciones de matrices por matrices en cada iteración es de $\mathcal{O}(n^2)$

- (e) Discuta bajo qué circunstancias puede ser conveniente igual usar el Método del Gradiente en vez de calcular explícitamente la solución óptima determinada en (c). Su discusión se debe basar en las propiedades del Método del Gradiente. Puede usar la versión del método que le resulte más adecuada (el “clásico” o el presentado como Método de Primer Orden). Justifique con precisión todo lo que argumente y explicita cualquier supuesto adicional que haga.

Solución: La elección del método a utilizar va a depender principalmente de 2 factores. Dependerá del tamaño del problema y de la precisión y velocidad que queramos obtener nuestra solución.

Si el tamaño del problema es muy grande y queremos obtener una solución ϵ -aproximada de manera rápida. Como calcular la solución de manera explícita toma un esfuerzo $\mathcal{O}(n^3)$ y el método del Gradiente toma un esfuerzo de $\mathcal{O}(n^2)$ por iteración. Entonces utilizaremos el método del Gradiente si la cantidad de iteraciones es significativamente menor a n es decir se debe cumplir que $\kappa \frac{1}{\epsilon} \ll n$ o $\kappa \log(\frac{1}{\epsilon}) \ll n$ dependiendo si desarrollamos el método simple de primer orden o el método del Gradiente respectivamente. Donde κ es una constante que toma en consideración el condicionamiento del Hessiano. Nota: Es importante considerar este factor ya que podemos notar que la función es

cuadrática y por lo tanto sus curvas de nivel son elipses y el método del Gradiente converge de manera más lenta, cuando las curvas de nivel son muy excéntricas, lo que ocurre cuando el Hessiano tiene un condicionamiento alto para funciones con curvas de nivel elípticas.

Por otro lado si el tamaño del problema es pequeño o manejable y/o se necesita abordar el problema de manera exacta la mejor opción sería calcular la solución óptima de manera explícita como esta descrita en c).

Ejercicio 3

Sea $f : \mathbb{R}^n \rightarrow \mathbb{R}$ una función convexa sobre $x \geq 0$, con $n \approx 40$, f queda definida de la siguiente forma:

$$f(x) = \frac{c}{\sum_{i=1}^n a_i x_i^2} + \sum_{j=1}^m \left(\sum_{i=1}^n b_{i,j} x_i - d_j \right)^2$$

Donde tanto como c , a_i , $b_{i,j}$, d_j son constantes positivas: Suponga que se quiere resolver el siguiente problema:

$$\min_{x \geq 0} f(x)$$

A partir de esta información, calcule $\nabla f(x)$ y $\nabla^2 f(x)$. Luego indique qué método elegiría para obtener el mínimo de la función, calculando además el esfuerzo computacional de dicho método (en notación asintótica $\mathcal{O}(\cdot)$).

Solución: Tenemos por un lado que

$$\frac{\partial f(x)}{\partial x_k} = \frac{-2ca_k x_k}{\left(\sum_{i=1}^n a_i x_i^2 \right)^2} + \sum_{j=1}^m 2b_{k,j} \left(\sum_{i=1}^n b_{i,j} x_i - d_j \right)$$

Ahora en cuanto a las segundas derivadas

$$\frac{\partial^2 f(x)}{\partial x_k^2} = \frac{4ca_k^2 x_k^2}{\left(\sum_{i=1}^n a_i x_i^2 \right)^3} - \frac{2ca_k}{\left(\sum_{i=1}^n a_i x_i^2 \right)^2} + \sum_{j=1}^m 2b_{k,j}^2$$

Si $l \neq k$

$$\frac{\partial^2 f(x)}{\partial x_k \partial x_l} = \frac{4ca_k x_k a_l x_l}{\left(\sum_{i=1}^n a_i x_i^2 \right)^3} + \sum_{j=1}^m 2b_{k,j} b_{l,j}$$

Luego definimos $\nabla f(x)$ y $\nabla^2 f(x)$:

$$\nabla f(x)^T = \left(\frac{\partial f(x)}{\partial x_1}, \dots, \frac{\partial f(x)}{\partial x_n} \right)$$
$$\nabla^2 f(x) = \begin{pmatrix} \frac{\partial^2 f(x)}{\partial x_1^2} & \cdots & \frac{\partial^2 f(x)}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f(x)}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f(x)}{\partial x_n^2} \end{pmatrix}$$

Ahora, cualquier método (Gradiente, Newton, etc.) es válida mientras este bien justificado. En cuanto esfuerzo computacional y a la cantidad de iteraciones.

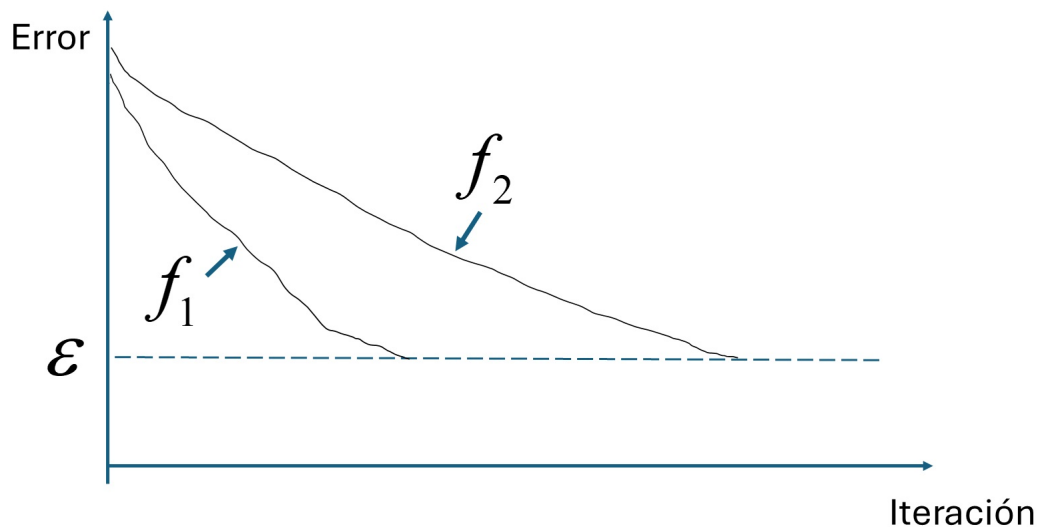
Tenemos para cada caso:

- Gradiente: $\mathcal{O}(\log(\frac{1}{\epsilon})n^2)$
- Newton: $\mathcal{O}(\log(\log(\frac{1}{\epsilon}))n^3)$

Ejercicio 4

Suponga que se desea minimizar una cierta función (no importa cuál) y usted “sabe” que esa función es convexa y diferenciable hasta el orden que uno quiera. Un grupo de ingenieros se pone a trabajar con los métodos de Gradiente y Newton en esta función, eligen puntos de partida, etc., lo típico que seguramente usted habría hecho.

El grupo le presenta a usted los resultados de la evolución del Método del Gradiente. Estos se muestran en el siguiente gráfico de la función objetivo a medida que avanzan las iteraciones:



¿Qué opinión le despiertan esos resultados? ¿Los consideraría correctos? Justifique con precisión su respuesta sobre la base de contenidos del curso.

Solución: Los resultados son incorrectos, ya que, por construcción, el método es monótono (esto cobra mayor importancia, considerando que la función es convexa). El error en los resultados puede deberse a diversos factores, entre los que se encuentra, por ejemplo, errores de cálculo en la etapa de *line search* (se obtiene un λ incorrecto).

Ejercicio 5

Queremos resolver el problema de minimización restringido $\min_{x \in \mathbb{R}^n} f(x)$, para una función f dos veces continuamente diferenciable. No asumiremos más hipótesis sobre f . Sabemos que podríamos usar el Método de Newton para esto pero, debe recordarse, para que este método funcione correctamente es necesario asegurarse de que en cualquier iteración k , la matriz $\nabla^2 f(x^k)$ sea definida positiva (para que la dirección de avance sea de descenso). Como f no es necesariamente convexa, eso no tendría necesariamente que cumplirse.

Antes de seguir, definiremos un concepto importante: Sea $Q \in \mathbb{R}^{n \times n}$ una matriz simétrica. Denotamos por q_{ij} el elemento fila i , columna j de Q . Decimos que Q es **diagonal dominante** si para cada fila i se tiene que $\sum_{j=1, j \neq i}^n |q_{ij}| < |q_{ii}|$.

Teorema: Si $Q \in \mathbb{R}^{n \times n}$ es una matriz simétrica diagonal dominante, entonces Q es definida positiva.

(**Nota:** La propiedad es sólo suficiente, hay matrices definidas positivas que no son diagonal dominantes.)

Proponemos el siguiente algoritmo, algo como una variante del Método de Newton:

Algoritmo de Newtoncito (para minimizar):

Data: $x^0 \in \mathbb{R}^n$, f , $k = 0$, $\epsilon > 0$.

```
while  $\|\nabla f(x^k)\|_2 > \epsilon$  do
  if  $\nabla^2 f(x^k)$  no es definida positiva then
    Sea  $B = \nabla^2 f(x^k)$ 
    for  $i = 1, \dots, n$  do
       $a_i = \sum_{j=1, j \neq i}^n |b_{ij}|$ 
      if  $a_i > |b_{ii}|$  then
         $b_{ii} = a_i + \eta$ 
      end
    end
  else
     $B = \nabla^2 f(x^k)$ 
  end
  Sea  $d^k = -B^{-1}\nabla f(x^k)$ 
  Resolver aproximadamente
      
$$\min_{\lambda \geq 0} f(x^k + \lambda d^k) \quad (\text{etapa de "linesearch"})$$

  Sea  $\lambda^k$  solución óptima de ese problema
  Sea  $x^{k+1} = x^k + \lambda^k d^k$ 
   $k = k + 1$ 
end
```

En este algoritmo, $\eta > 0$ es un número dado, y b_{ij} es el elemento fila i , columna j de la matriz B .

- (a) Explique qué es lo que hace este algoritmo y por qué podría ser buena idea hacer lo que ahí se propone. Justifique con precisión su respuesta y no se extienda en más de 5-6 líneas.

Solución: El algoritmo apunta a asegurar que el Hessiano sea definido positivo, para aprovechar las propiedades del Método de Newton. En ese sentido, si $\nabla^2 f(x^k)$ resulta no serlo, lo que hace es ajustar los elementos diagonales, de modo que la matriz sea diagonal dominante. Una vez hecho esto, por el teorema mencionado en el enunciado, entonces $\nabla^2 f(x^k)$ es definido positivo. Sería una buena idea utilizar este algoritmo, ya que permite trabajar sobre una clase más amplia de funciones, que puede que no cumplan necesariamente las características claves del Método de Newton. Es decir, por medio de la transformación de los elementos diagonales, se garantiza que el Hessiano será definido positivo, y, con ello, el método terminará convergiendo, dado que la dirección será de descenso.

- (b) ¿Qué cree usted que podría pasar con las propiedades de convergencia del algoritmo,

incluyendo la complejidad (es decir, el orden del número de iteraciones necesarias para llegar a un error dado), comparadas con la del Método de Newton? Justifique con precisión su respuesta y no se extienda en más de 5-6 líneas.

Solución: La pregunta, como menciona el enunciado, se refiere a la complejidad en número de iteraciones para convergencia, no la complejidad computacional como tal. Dicho eso, se tiene que posiblemente podría no haber convergencia cuadrática, según el grado de perturbación que se induzca en la matriz producto de una potencial transformación a su diagonal. En efecto, si los elementos diagonales resultan ser significativamente diferentes de los originales en sucesivas iteraciones de $\nabla^2 f(x^k)$, entonces es posible que se pierda la convergencia cuadrática, siendo, entonces, peor que el Método de Newton.

- (c) Aún suponiendo que en una iteración k , la matriz $\nabla^2 f(x^k)$ no sea definida positiva, ¿Qué característica cree usted que debería tener $\nabla^2 f(x^k)$ para que la convergencia del algoritmo Newtoncito sea prácticamente igual a la del Método de Newton original (es decir, que hagan, aproximadamente, el mismo número de iteraciones para un mismo caso de problema.)? Justifique con precisión su argumento.

Solución: Para que la convergencia del algoritmo Newtoncito sea prácticamente igual a la del Método de Newton, sabiendo que $\nabla^2 f(x^k)$ no es definida positiva, podría ser que $\nabla^2 f(x^k)$ tenga pocos valores propios negativos, y de serlos, cercanos a cero. Alternativamente, debería tener valores propios positivos, casi en su totalidad. De este modo, $\nabla^2 f(x^k)$ estaría muy cerca de ser definida positiva, por lo que las correcciones se ejecutarían pocas veces.

- (d) Supongamos que el algoritmo Newtoncito tiene éxito y eventualmente comienza a converger a un mínimo local de la función f . Argumente que, eventualmente, el algoritmo usará una etapa normal de Newton y no será necesario hacer la corrección de las líneas 3 a 8 del pseudocódigo de arriba.

Solución: Dado que Newtoncito está convergiendo, conforme siguen las iteraciones, el Hessiano se vuelve definido positivo, ya que se está acercando al mínimo. También, va mejorando el condicionamiento de la matriz, volviéndose estable con el paso del algoritmo. Es así como, entrada en la etapa final, derechamente no es necesario corregirlo con las líneas 3 a 8 del algoritmo.